

Hero Cycles Pricing Engine

Full-Stack Engineer (Fresher) Hiring Assignment

Prepared By

Harshit

B.Tech Computer Science Engineering
Vellore Institute of Technology (VIT)

Assignment Objective

Design and develop a pricing engine for Hero Cycles that enables sales teams to manage bicycle components, maintain price histories, create cycle configurations, and instantly generate accurate pricing breakdowns.

Submission Contents

1. Problem Analysis
2. Questions & Assumptions
3. Functional Requirements
4. Solution Design
5. Data Model
6. System Architecture
7. API Design
8. Pseudocode
9. User Interface Design
10. Testing Strategy
11. Future Enhancements
12. AI Usage & Prompt Log

Submission Date: June 2026

Technology Stack: React.js, Node.js, Express.js, MongoDB

Github Link: <https://github.com/vostok2003/hero-cycles-pricing-engine>

Live Link: <https://hero-cycles-pricing-engine.vercel.app/login>

Salesman account:

username: harshit@gmail.com

Password: 123456

Admin account:

Username: tanvi@gmail.com password: 123456

Problem Analysis

Hero Cycles manufactures and sells bicycles that are assembled using multiple components such as frames, tyres, gear sets, seats, and brakes. Since component prices change periodically, maintaining accurate bicycle pricing through spreadsheets has become inefficient and error-prone.

The organization requires a centralized pricing engine that allows users to manage components, track price revisions, create bicycle configurations, and automatically calculate the total cost of a bicycle using the latest available component prices.

The system should also provide a detailed component-wise cost breakdown and maintain historical pricing information for future reference and auditing purposes.

The primary objective is to eliminate manual price calculations, reduce pricing errors, and enable sales teams to generate accurate quotations instantly.

Questions Considered

While analyzing the problem statement, I considered the following questions:

1. What components (frames, tyres, gear sets, seats, etc.) are required to build a bicycle configuration?
2. How should the system store and track component price changes over time?
3. How can the system determine the currently active price of a component?
4. How should bicycle configurations be created and managed?
5. Can the same component be used in multiple bicycle configurations?
6. How can component prices be managed from a centralized location instead of spreadsheets?
7. How should the system generate a component-wise pricing breakdown?
8. How can the total bicycle price be calculated automatically using the latest component prices?

Assumption

1. The latest active price of a component will be used for all pricing calculations.
2. Historical price records will be preserved and will not be overwritten when prices are updated.
3. The system will initially use a limited set of sample components and bicycle configurations for demonstration purposes.
4. Mock/synthesized data will be used where real business data is unavailable.
5. Every bicycle configuration must contain a set of mandatory core components such as a frame, tyre set, and gear set.
6. A component can be used across multiple bicycle configurations.
7. The primary goal of the system is to provide accurate and up-to-date pricing information to the sales team.
8. The scope of this assignment is limited to component management, price tracking, configuration management, and pricing calculation. Features such as taxation, discounts, inventory tracking, and quotation management are considered future enhancements.

System design decisions

1. Centralized Price Management

Component prices are managed from a central location instead of being stored within individual bicycle configurations. This ensures consistency and prevents duplicate pricing data.

2. Price History Tracking

Whenever a component price is updated, a new price record is created instead of overwriting the previous one. This preserves historical pricing information and allows tracking of price changes over time.

3. Dynamic Price Calculation

Configuration prices are calculated using the latest active component prices rather than storing fixed totals. This ensures that pricing always reflects current component costs.

4. Configuration-Based Modeling

A bicycle configuration is represented as a collection of components. This approach allows components to be reused across multiple configurations.

5. Role-Based Access

Administrators can manage components and pricing information, while sales users can create and modify bicycle configurations and view pricing details.

6. Mandatory Core Components

Certain components such as frame, tyres, and gear sets are treated as mandatory parts of a valid bicycle configuration. These components cannot be removed but may be replaced with alternatives from the same category.

Functional Requirements

1. Component Management

- The system shall allow administrators to create new bicycle components.
- The system shall allow administrators to update component details.
- The system shall allow administrators to view all available components.
- Components shall be categorized (e.g., Frame, Tyre, Gear Set, Seat, Brake).

2. Price Management

- The system shall allow administrators to update component prices.
- The system shall maintain a record of previous price revisions.
- The system shall display the latest active price for each component.
- The system shall display the date of the most recent price update.

3. Configuration Management

- The system shall allow users to create bicycle configurations.
- The system shall allow components to be added to a configuration.
- The system shall allow optional components to be removed from a configuration.
- The system shall allow compatible components to be replaced with alternatives from the same category.
- A component may be used in multiple bicycle configurations.

4. Pricing Engine

- The system shall automatically calculate the total price of a bicycle configuration.
- The system shall use the latest active prices during calculation.
- The system shall generate a detailed component-wise price breakdown.
- Any component change within a configuration shall automatically update the total price.

5. User Access

- Administrators shall be able to manage components and pricing information.
- Sales users shall be able to create and modify bicycle configurations.
- Sales users shall be able to view configuration pricing and cost breakdowns.

6. Dashboard & Reporting

- The system shall display all available bicycle configurations.
- The system shall display pricing information in an easily understandable format.
- The system shall provide visibility into recently updated component prices.

Use Cases

Use Case 1: Add a New Component

Actor: Admin

Description:

The administrator adds a new bicycle component to the system.

Flow:

1. Admin navigates to the Component Management page.
2. Admin enters component details.
3. Admin saves the component.
4. System stores the component and makes it available for future configurations.

Use Case 2: Update Component Price

Actor: Admin

Description:

The administrator updates the price of an existing component.

Flow:

1. Admin selects a component.
2. Admin enters the updated price.
3. System records the new price.
4. System marks the latest price as active.
5. Updated pricing becomes available for all configurations.

Use Case 3: Create a Bicycle Configuration

Actor: Salesperson

Description:

The salesperson creates a bicycle configuration using available components.

Flow:

1. Salesperson enters a configuration name.

2. Salesperson selects components from different categories.
3. System validates the configuration.
4. System saves the configuration.

Use Case 4: Modify a Configuration

Actor: Salesperson

Description:

The salesperson customizes a bicycle configuration.

Flow:

1. Salesperson opens an existing configuration.
2. Salesperson replaces optional components with alternatives.
3. System updates the configuration.
4. Pricing is recalculated automatically.

Use Case 5: Generate Pricing Breakdown

Actor: Salesperson

Description:

The salesperson views the detailed cost of a bicycle configuration.

Flow:

1. Salesperson selects a configuration.
2. System retrieves the latest active prices.
3. System displays component-wise pricing.
4. System calculates and displays the total configuration price.

Use Case 6: View Recent Price Updates

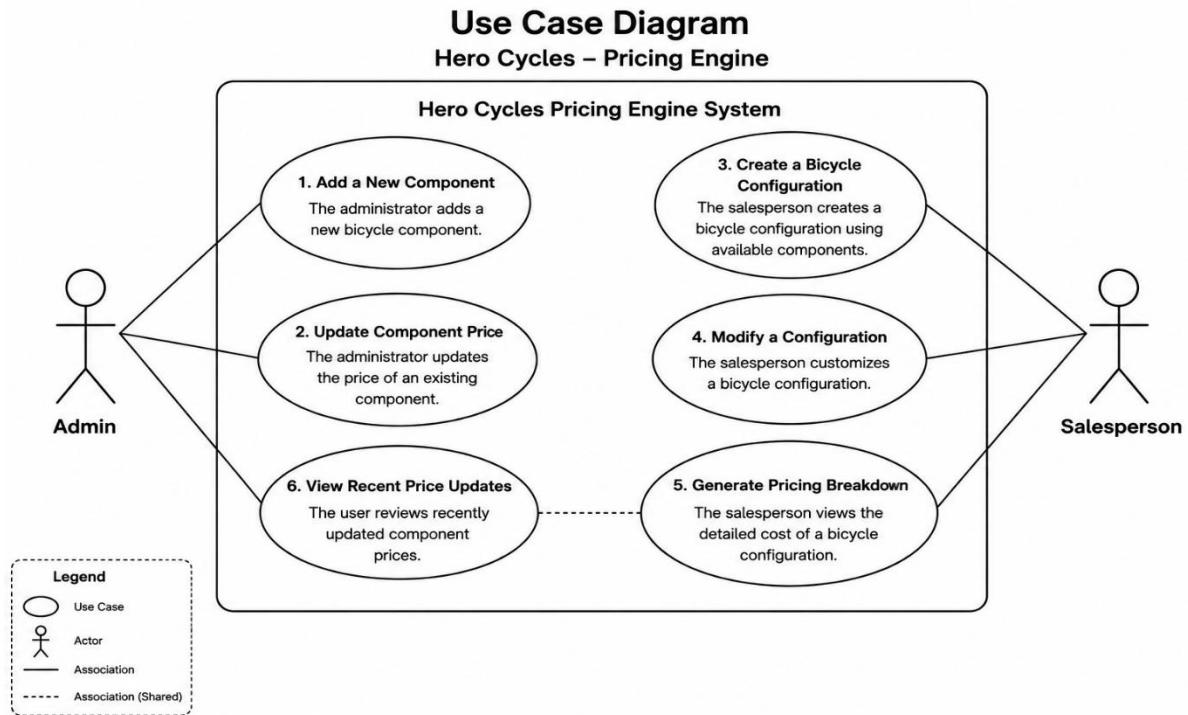
Actor: Admin / Salesperson

Description:

The user reviews recently updated component prices.

Flow:

1. User opens the pricing dashboard.
2. System displays recent price changes.
3. User reviews updated pricing information.



Solution Design

To address the pricing and configuration management challenges, the proposed solution follows a centralized component-based architecture.

1. Centralized Component Repository

All bicycle components such as frames, tyres, gear sets, seats, and brakes will be stored in a centralized repository.

Each component will contain:

- Component Name
- Category
- Current Price
- Last Updated Date

This ensures that pricing information remains consistent across all bicycle configurations.

2. Price Management System

Administrators will be able to update component prices from a dedicated management interface.

Instead of modifying configuration prices directly, component prices will be maintained centrally. Whenever a price is updated, the latest value becomes available throughout the system.

This eliminates manual updates across multiple configurations and reduces pricing errors.

3. Configuration Builder

Sales users will be able to create bicycle configurations by selecting components from different categories.

A configuration represents a complete bicycle assembled from multiple components.

Examples:

- Mountain Bike
- Road Bike
- Hybrid Bike

Core components such as frames, tyres, and gear sets are mandatory, while optional components may be customized based on business requirements.

4. Dynamic Pricing Engine

The pricing engine is responsible for calculating the total cost of a bicycle configuration.

The calculation process follows these steps:

1. Retrieve all components associated with a configuration.
2. Fetch the latest active price of each component.
3. Generate a component-wise price breakdown.
4. Calculate the total configuration cost.
5. Display the final price to the user.

Any change in component selection or component pricing automatically reflects in the final configuration price.

5. Role-Based Access Control

Two user roles are supported:

Administrator

- Manage components
- Update prices
- View pricing information

Salesperson

- Create configurations
- Modify configurations
- View component breakdown

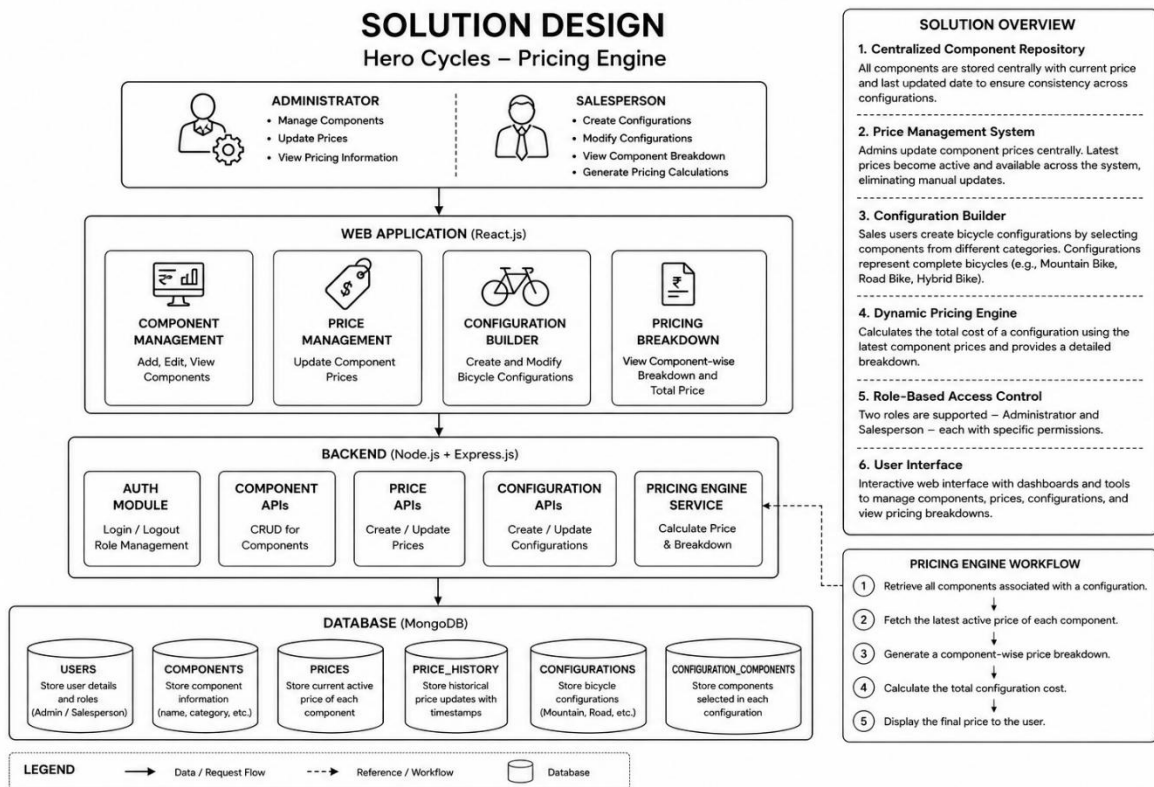
- Generate pricing calculations

6. User Interface

The system provides an interactive web interface consisting of:

- Component Management Dashboard
- Price Management Screen
- Configuration Builder
- Pricing Breakdown View

The interface is designed to minimize manual calculations and improve pricing visibility for sales teams.



Database Design

The system uses MongoDB as the primary database. The database is designed to support component management, price tracking, bicycle configurations, and dynamic pricing calculations.

Collections Overview

1. Users

Stores user information and role details.

Field	Type
_id	ObjectId
name	String
email	String
password	String
role	String (Admin / Salesperson)

2. Components

Stores all bicycle components available in the system.

Field	Type
_id	ObjectId
componentName	String
category	String
currentPrice	Number
lastUpdatedDate	Date
isMandatory	Boolean

3. PriceHistory

Stores historical price updates of components.

Field	Type
_id	ObjectId
componentId	ObjectId
oldPrice	Number
newPrice	Number
updatedBy	ObjectId
effectiveDate	Date

4. Configurations

Stores bicycle configuration information.

Field	Type
_id	ObjectId
configurationName	String
description	String
createdBy	ObjectId

5. ConfigurationComponents

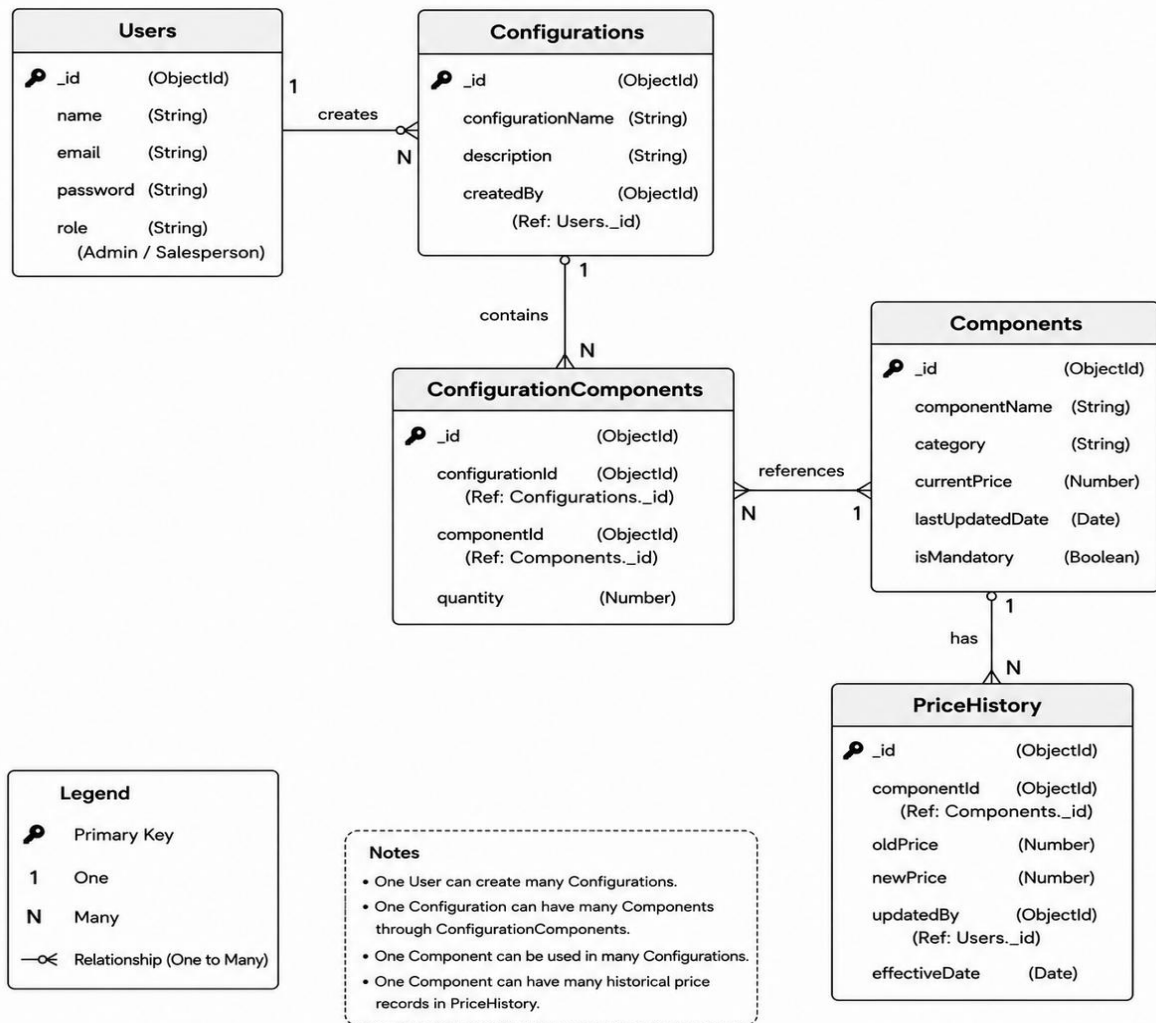
Stores the mapping between configurations and components.

Field	Type
_id	ObjectId
configurationId	ObjectId
componentId	ObjectId
quantity	Number

Entity Relationship Diagram

Entity Relationship Diagram

Hero Cycles – Pricing Engine



API Design

The backend exposes REST APIs for authentication, component management, configuration management, and pricing calculations.

Authentication APIs

Register User

POST /api/auth/register

Request Body:

```
{  
  "name": "Harshit",
```

```
"email": "harshit@gmail.com",  
"password": "password123",  
"role": "salesperson"  
}
```

Login User

POST /api/auth/login

Request Body:

```
{  
  "email": "harshit@gmail.com",  
  "password": "password123"  
}
```

Component APIs

Get All Components

GET /api/components

Purpose:

Retrieve all available bicycle components.

Create Component

POST /api/components

Request Body:

```
{  
  "componentName": "MTB Tyre",  
  "category": "Tyre",  
  "currentPrice": 500,  
  "isMandatory": true  
}
```

Update Component

PUT /api/components/:id

Purpose:

Update component information.

Delete Component

DELETE /api/components/:id

Purpose:

Remove a component.

Price Management APIs

Update Component Price

POST /api/prices/update

Request Body:

```
{  
  "componentId": "123",  
  "newPrice": 650  
}
```

Purpose:

Update component price and create a PriceHistory record.

Get Price History

GET /api/prices/history/:componentId

Purpose:

Retrieve all historical price updates for a component.

Configuration APIs

Create Configuration

POST /api/configurations

Request Body:

```
{  
  "configurationName": "Mountain Bike",  
  "description": "Premium MTB"  
}
```

Add Component to Configuration

POST /api/configurations/:id/components

Request Body:

```
{  
  "componentId": "456",
```

```
"quantity": 2
}
```

Update Configuration

PUT /api/configurations/:id

Purpose:
Modify an existing bicycle configuration.

Get Configuration Details

GET /api/configurations/:id

Purpose:
Retrieve configuration information and selected components.

Pricing Engine APIs

Generate Pricing Breakdown

GET /api/pricing/:configurationId

Response:

```
{
  "configurationName": "Mountain Bike",
  "components": [
    {
      "name": "Frame A",
      "price": 3000
    },
    {
      "name": "MTB Tyre",
      "price": 500
    }
  ],
  "totalPrice": 3500
}
```

Purpose:
Generate component-wise pricing breakdown and calculate total configuration cost.

Dashboard APIs

Get Recent Price Updates

GET /api/dashboard/recent-price-updates

Purpose:

Retrieve recently updated component prices.

Get Dashboard Summary

GET /api/dashboard/summary

Purpose:

Retrieve total components, configurations, and recent updates.

Pseudocode

The following pseudocode describes the core functionality of the Hero Cycles Pricing Engine system.

1. User Authentication

FUNCTION Login(email, password)

 FIND user using email

 IF user does not exist THEN

 RETURN "Invalid Credentials"

 END IF

 VERIFY password

 IF password is incorrect THEN

 RETURN "Invalid Credentials"

 END IF

 GENERATE JWT Token

 RETURN User Information and Token

END FUNCTION

2. Add New Component

FUNCTION AddComponent(componentData)

 VALIDATE component information

 CREATE component record

 STORE component in database

 RETURN "Component Added Successfully"

END FUNCTION

3. Update Component Price

FUNCTION UpdateComponentPrice(componentId, newPrice, adminId)

 FIND component using componentId

 oldPrice ← component.currentPrice

 CREATE priceHistoryRecord

 componentId

 oldPrice

 newPrice

 updatedBy = adminId

 effectiveDate = Current Date

 SAVE priceHistoryRecord

UPDATE component.currentPrice = newPrice

UPDATE component.lastUpdatedDate

SAVE component

RETURN "Price Updated Successfully"

END FUNCTION

4. Create Bicycle Configuration

FUNCTION CreateConfiguration(configurationData)

VERIFY mandatory categories exist

Frame

Tyre

Gear Set

IF any mandatory category is missing THEN

RETURN "Configuration Invalid"

END IF

CREATE configuration

SAVE configuration

FOR EACH selected component

CREATE configuration-component mapping

END FOR

RETURN "Configuration Created Successfully"

END FUNCTION

5. Modify Configuration

FUNCTION ModifyConfiguration(configurationId, updatedComponents)

FIND configuration

VERIFY mandatory categories remain present

IF mandatory component removed THEN

RETURN "Operation Not Allowed"

END IF

UPDATE configuration components

SAVE configuration

RETURN "Configuration Updated Successfully"

END FUNCTION

6. Generate Pricing Breakdown

FUNCTION GeneratePricingBreakdown(configurationId)

FETCH configuration

FETCH all associated components

totalPrice $\leftarrow$ 0

FOR EACH component

 unitPrice $\leftarrow$ component.currentPrice

 subtotal $\leftarrow$ unitPrice $\times$ quantity

 ADD subtotal to totalPrice

 STORE component breakdown

END FOR

RETURN

 Configuration Details

 Component Breakdown

 Total Price

END FUNCTION

7. Dynamic Pricing Engine

FUNCTION CalculateConfigurationPrice(configurationId)

 configuration $\leftarrow$ GetConfiguration(configurationId)

 components $\leftarrow$ GetConfigurationComponents(configurationId)

totalPrice $\leftarrow$ 0

pricingBreakdown $\leftarrow$ Empty List

FOR EACH component

latestPrice $\leftarrow$ component.currentPrice

subtotal $\leftarrow$ latestPrice $\times$ quantity

totalPrice $\leftarrow$ totalPrice + subtotal

ADD component details to pricingBreakdown

END FOR

RETURN

pricingBreakdown

totalPrice

END FUNCTION

8. Dashboard Summary

FUNCTION GetDashboardSummary()

totalComponents $\leftarrow$ COUNT Components

totalConfigurations $\leftarrow$ COUNT Configurations

recentPriceUpdates $\leftarrow$ FETCH Recent Price History Records

RETURN Dashboard Data

END FUNCTION

9. Complete System Workflow

START

User Login

IF User Role = Admin THEN

Manage Components

Update Component Prices

View Price History

ELSE IF User Role = Salesperson THEN

Create Configuration

Modify Configuration

Generate Pricing Breakdown

END IF

Pricing Engine Fetches Latest Component Prices

System Calculates Component Subtotals

System Calculates Total Configuration Cost

Display Component-wise Pricing Breakdown

Display Final Price

END

Explanation

The pricing engine follows a centralized pricing approach. Component prices are maintained by administrators and stored in a common repository. Whenever a salesperson creates or modifies a bicycle configuration, the system automatically retrieves the latest component prices, generates a detailed component-wise breakdown, and calculates the final bicycle price. All price updates are tracked through the Price History module to maintain transparency and pricing consistency across configurations.

UI/UX Design

The user interface is designed to provide a simple and efficient workflow for administrators and sales users. The primary focus is to minimize manual calculations and provide quick access to pricing information.

Design Principles

- Simple and intuitive navigation
- Minimal number of steps for pricing calculations
- Clear component-wise price visibility
- Responsive and user-friendly interface
- Real-time pricing updates
- Role-based user experience

Screen 1: Authentication

Purpose:

Allow users to securely access the system.

Features:

- Login Form
- Role-based access (Admin / Salesperson)

Screen 2: Role-Based Dashboard

Purpose:

Provide users with a quick overview of information and actions relevant to their role.

Administrator Dashboard

Features:

- Total Components
- Total Configurations
- Recent Price Updates
- Quick Navigation Cards
- Component Management Access
- Price Management Access
- Price History Monitoring

Salesperson Dashboard

Features:

- Total Configurations
- Recently Modified Configurations
- Recent Price Updates
- Configuration Builder Access
- Configuration Management Access
- Pricing Breakdown Access

The dashboard content changes dynamically based on the authenticated user's role to reduce unnecessary information and improve usability.

Screen 3: Component Management

Purpose:

Manage bicycle components.

Features:

- Add Component
- Edit Component
- Delete Component
- Component Category Filtering
- Search Components

Screen 4: Price Management

Purpose:

Manage component pricing.

Features:

- View Current Price
- Update Price
- View Price History
- Last Updated Information

Screen 5: Configuration Builder

Purpose:

Create and modify bicycle configurations.

Features:

- Create Configuration
- Select Components
- Replace Components
- Manage Quantities
- Save Configuration

Screen 6: Pricing Breakdown

Purpose:

Display detailed pricing information.

Features:

- Component-wise Cost Breakdown
- Latest Component Prices
- Total Configuration Cost
- Automatic Recalculation

User Flow

Administrator

Login

↓

Administrator Dashboard

↓

Manage Components

↓

Update Component Prices

↓

View Price History

Salesperson

Login



Salesperson Dashboard



Create / Modify Configuration



Generate Pricing Breakdown



View Final Price

Hero Cycles
Pricing Engine

tanvi gumber
Admin

Dashboard

Components

Price Management

Price History

Logout

Admin Portal

admin tanvi@gmail.com

Price History

Complete log of all component price changes.

All Price Updates

3 total records

Component	Category	Old Price	New Price	Change	Updated By	Effective Date
KG-9000	Frame	₹10,000.00	₹11,000.00	+₹1,000.00	tanvi gumber	19 Jun 2026, 02:00 pm
MTX-500	Frame	₹700.00	₹4,000.00	+₹3,300.00	tanvi gumber	19 Jun 2026, 01:42 pm
RF-500	Frame	₹700.00	₹1,200.00	+₹500.00	Anjali yadav	19 Jun 2026, 01:08 pm

Price Management

Update component prices. All changes are tracked automatically.

Update Component Price

Select Component

-- Choose a component --

New Price (₹)

Enter new price

Update Price

Recent Price Changes

Component	Category	Old Price	New Price	Change	Updated By	Effective Date
KG-9000	Frame	₹10,000.00	₹11,000.00	+₹1,000.00	tanvi gumber	19 Jun 2026, 02:00 pm
MTX-500	Frame	₹700.00	₹4,000.00	+₹3,300.00	tanvi gumber	19 Jun 2026, 01:42 pm
RF-500	Frame	₹700.00	₹1,200.00	+₹500.00	Anjali yadav	19 Jun 2026, 01:08 pm

Hero Cycles

Pricing Engine

T

tanvi gumber

Admin

Dashboard

Components

Price Management

Price History

Logout

Admin Portal

admin | tanvi@gmail.com

Components

Manage bicycle components and their pricing.

Q Search components...

All

Component Name	Category	Current Price	Mandatory	Last Updated	Actions
wide1000	Frame	₹800.00	Optional	19 Jun 2026	Edit Delete
wide-500	Tyre	₹600.00	Mandatory	19 Jun 2026	Edit Delete
Regular	Brake	₹200.00	Mandatory	19 Jun 2026	Edit Delete
UG-80	Brake	₹600.00	Optional	19 Jun 2026	Edit Delete
KG-9000	Frame	₹11,000.00	Optional	19 Jun 2026	Edit Delete
premium	Seat	₹800.00	Optional	19 Jun 2026	Edit Delete
RF-500	Frame	₹1,200.00	Mandatory	19 Jun 2026	Edit Delete
RF-200	Gear Set	₹500.00	Mandatory	19 Jun 2026	Edit Delete
regular	Seat	₹400.00	Mandatory	19 Jun 2026	Edit Delete

Hero Cycles

Pricing Engine

T

tanvi gumber

Admin

Dashboard

Components

Price Management

Price History

Logout

Admin Portal

admin | tanvi@gmail.com

Admin Dashboard

Welcome back, tanvi gumber. Here's your system overview.

TOTAL COMPONENTS

10

Across all categories

TOTAL CONFIGURATIONS

2

Bicycle builds created

PRICE UPDATES (7D)

3

Changes in the last week

Components by Category

2

Brake

4

Frame

1

Gear Set

2

Seat

1

Tyre

Recent Price Updates

KG-9000

Frame · Updated by tanvi gumber

₹10,000.00 ₹11,000.00

19 Jun 2026

MTX-500

Frame · Updated by tanvi gumber

₹700.00 ₹4,000.00

19 Jun 2026

RF-500

Frame · Updated by Anjali yadav

₹700.00 ₹1,200.00

19 Jun 2026

Hero Cycles

Pricing Engine

Sign In

Register

Email

you@example.com

Password

Sign In

Hero Cycles Pricing Engine © 2024

Hero Cycles
Pricing Engine

saksham
Salesperson

Dashboard

Configuration Builder

Pricing Breakdown

Logout

Sales Portal

salesperson saksham@gmail.com

Pricing Breakdown

Select a configuration to see the full component cost breakdown.

Select Configuration

hybrid rover-689

hybrid rover-689

Created by saksham - Calculated 19 Jun 2026, 02:38 pm

Refresh

COMPONENT	CATEGORY	QUANTITY	UNIT PRICE	SUBTOTAL
MTX-500	Required	Frame	1	₹4,000.00
wide-500	Required	Tyre	2	₹600.00
RF-200	Required	Gear Set	1	₹500.00
regular	Required	Seat	1	₹400.00
UG-80	Required	Brake	1	₹600.00
Grand Total				₹6,700.00

Hero Cycles
Pricing Engine

saksham
Salesperson

Dashboard

Configuration Builder

Pricing Breakdown

Logout

Sales Portal

salesperson saksham@gmail.com

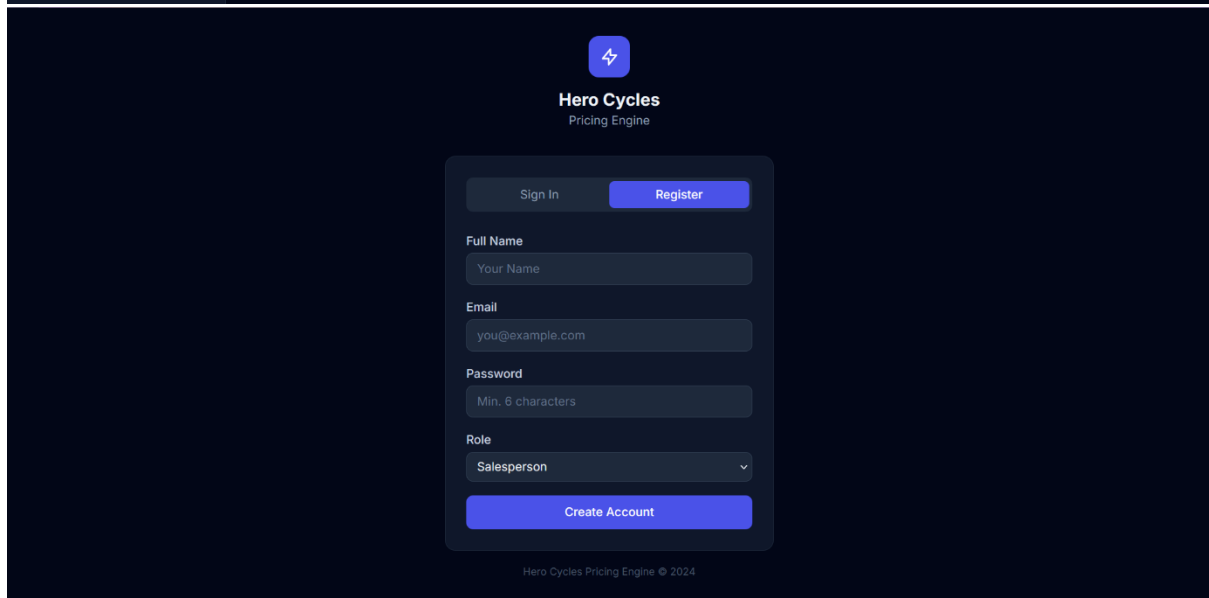
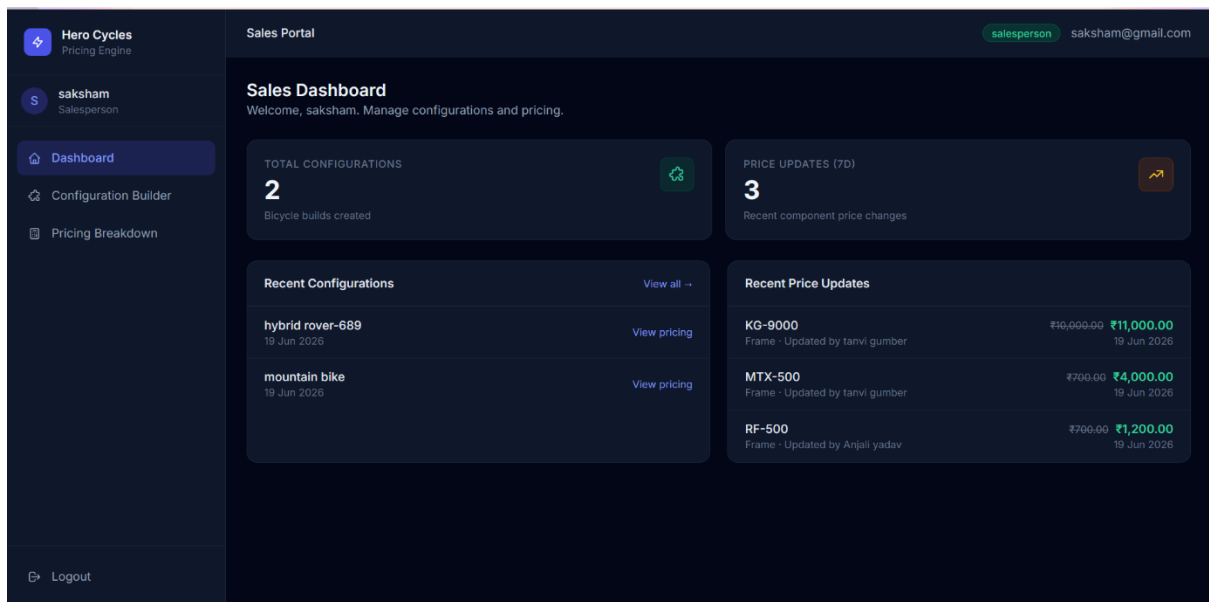
Configuration Builder

Build and manage bicycle configurations.

+ New Configuration

Q Search configurations...

CONFIGURATION NAME	DESCRIPTION	CREATED BY	CREATED	ACTIONS
hybrid rover-689	—	saksham	19 Jun 2026	Pricing Edit Delete
mountain bike	—	harshit	19 Jun 2026	Pricing Edit Delete



Testing

To ensure the reliability, accuracy, and usability of the Hero Cycles Pricing Engine, comprehensive testing was performed across all major modules of the application. The testing process focused on validating authentication, role-based access control, component management, price tracking, configuration management, dashboard functionality, and dynamic pricing calculations.

The objective of testing was to verify that the application behaves as expected under different user actions and business scenarios while maintaining accurate pricing information throughout the system.

Authentication Testing

Authentication testing was performed to verify secure user access and role-based authorization.

Test Scenarios

Test Case	Expected Result
Register a new user	User account created successfully
Login with valid credentials	User authenticated successfully
Login with invalid credentials	Appropriate error message displayed
Access protected routes without login	Access denied
Admin accessing salesperson functionality	Restricted according to permissions
Salesperson accessing admin functionality	Restricted according to permissions

Result

Authentication and authorization mechanisms worked successfully and ensured secure access based on user roles.

Component Management Testing

Component management functionality was tested to verify the complete lifecycle of bicycle components.

Test Scenarios

Test Case	Expected Result
Add a new component	Component added successfully
Edit component details	Component information updated
Delete a component	Component removed successfully
Search for a component	Matching results displayed
Filter components by category	Relevant components displayed

Result

All CRUD operations for component management functioned correctly and data was persisted successfully in the database.

Price Management Testing

Price management testing ensured that component prices could be updated while preserving historical pricing information.

Test Scenarios

Test Case	Expected Result
Update component price	New price saved successfully
Create price history record after update	Price history generated automatically
Perform multiple price updates	Multiple historical records stored correctly
View price history	Complete price history displayed accurately

Result

Price updates were successfully reflected across the system, and historical pricing records were maintained without data loss.

Configuration Management Testing

Configuration management functionality was tested to ensure bicycle configurations could be created and modified correctly.

Test Scenarios

Test Case	Expected Result
Create a new configuration	Configuration created successfully
Add mandatory components	Configuration saved successfully
Remove mandatory component	Validation error generated
Replace component with another component of same category	Configuration updated successfully
Modify existing configuration	Changes saved correctly

Result

The system successfully enforced configuration rules and ensured that mandatory components remained part of valid bicycle configurations.

Pricing Engine Testing

The pricing engine is the core module of the application. Testing was performed to verify calculation accuracy and dynamic pricing behavior.

Test Scenarios

Test Case	Expected Result
Generate pricing breakdown	Detailed component-wise pricing displayed
Update component quantity	Subtotals recalculated correctly
Update component price	Latest price reflected automatically
Calculate total configuration cost	Accurate total generated

Sample Validation

Frame = ₹4000 × 1 = ₹4000

Tyre = ₹600 × 2 = ₹1200

Gear Set = ₹500 × 1 = ₹500

Seat = ₹400 × 1 = ₹400

Brake = ₹600 × 1 = ₹600

Expected Total = ₹6700

Generated Total = ₹6700

Result

The pricing engine correctly calculated component subtotals and generated accurate configuration pricing using the latest available component prices.

Dashboard Testing

Dashboard functionality was tested to ensure role-specific information was displayed correctly.

Test Scenarios

Test Case	Expected Result
Load Administrator Dashboard	Administrative statistics displayed
Load Salesperson Dashboard	Salesperson-specific information displayed
View recent price updates	Latest updates displayed correctly
View configuration statistics	Accurate counts displayed

Result

Role-based dashboards displayed relevant information and navigation options according to the authenticated user's permissions.

End-to-End Workflow Testing

A complete business workflow was executed to validate integration between all application modules.

Workflow

1. Administrator logs into the system.
2. Administrator creates bicycle components.
3. Administrator updates component prices.
4. Price history records are generated automatically.
5. Salesperson logs into the system.
6. Salesperson creates a bicycle configuration.
7. Salesperson selects bicycle components.
8. Pricing engine retrieves the latest component prices.
9. System generates a detailed pricing breakdown.
10. Final bicycle price is calculated and displayed.

Result

The complete workflow executed successfully without errors. Data consistency was maintained across all modules, and pricing calculations remained accurate throughout the process.

Conclusion

Testing confirmed that all major functionalities of the Hero Cycles Pricing Engine operate as expected. Authentication, role-based access control, component management, price tracking, configuration management, dashboard reporting, and dynamic pricing calculations were validated successfully. The application consistently generated accurate pricing breakdowns while maintaining historical pricing information and enforcing business rules across bicycle configurations.

Future Enhancements

While the current implementation successfully addresses the core requirements of component management, price tracking, configuration management, and dynamic pricing calculations, several enhancements can be introduced to further improve the system and make it more suitable for large-scale business operations.

1. Tax and Discount Engine

The current system calculates pricing using component costs only. Future versions can support:

- GST and tax calculations
- Seasonal discounts

- Promotional offers
- Dealer-specific discounts
- Corporate pricing policies

This would enable the generation of more accurate customer-facing quotations.

2. PDF Quotation Generation

The system can be extended to generate professional quotations containing:

- Customer details
- Bicycle configuration details
- Component-wise pricing breakdown
- Taxes and discounts
- Final payable amount

Quotations can be exported and shared as PDF documents.

3. Inventory Management Integration

Future versions can integrate inventory tracking to provide:

- Available stock information
- Low-stock alerts
- Automatic stock deduction after order confirmation
- Inventory forecasting

This would help align pricing and configuration management with actual product availability.

4. Advanced Analytics Dashboard

Additional reporting and analytics features can be introduced, including:

- Most frequently used components
- Most popular bicycle configurations
- Price change trends
- Configuration profitability analysis
- Sales performance metrics

These insights would support better business decision-making.

5. Category Management

Currently, component categories are predefined. Future versions can allow administrators to:

- Create new categories
- Modify existing categories
- Disable unused categories

This would improve system flexibility and scalability.

6. Multi-Branch Support

Organizations operating across multiple locations may require:

- Branch-specific pricing
- Branch-specific inventories
- Branch-level configuration management
- Regional pricing policies

This would allow the system to support distributed business operations.

7. Approval Workflow for Price Changes

A controlled approval mechanism can be introduced where:

- Price changes are proposed by one user
- Approved by authorized managers
- Automatically activated after approval

This would improve governance and reduce pricing errors.

8. Customer and Dealer Management

The application can be expanded to maintain:

- Customer profiles
- Dealer information
- Purchase history
- Quotation history

This would transform the system into a more complete business management platform.

9. Notification System

Future releases may include automated notifications for:

- Component price updates
- Configuration changes
- Approval requests
- Important system activities

Notifications can be delivered through email or in-application alerts.

10. Audit Logs and Activity Tracking

To improve accountability and traceability, the system can maintain:

- User activity logs
- Configuration modification history
- Price update history
- Administrative actions

This would be especially useful in enterprise environments.

11. Cloud Deployment and CI/CD

The application can be enhanced with modern deployment practices such as:

- Docker containerization
- CI/CD pipelines
- Automated testing
- Cloud deployment on AWS, Azure, or GCP

This would improve maintainability, scalability, and reliability.

12. Mobile-Friendly Experience

A dedicated mobile experience can be introduced to allow sales representatives to:

- Create configurations on the go
- Access pricing information instantly
- Generate quotations from mobile devices

This would improve accessibility and field productivity.

Conclusion

The current Hero Cycles Pricing Engine establishes a strong foundation for centralized pricing and configuration management. The enhancements outlined above provide a roadmap for evolving the application into a comprehensive enterprise-grade platform capable of supporting large-scale bicycle manufacturing and sales operations.

AI Usage & Prompt Log

AI-assisted tools were used during the brainstorming, documentation, system design, architecture planning, database design, API design, UI/UX planning, and implementation phases of this assignment.

The final solution, architecture decisions, implementation choices, and business rules were reviewed and adapted based on the project requirements.

Representative Prompts Used

Problem Analysis

- Analyze the Hero Cycles pricing engine problem statement and identify the core business requirements.
- What questions should be considered before designing a bicycle pricing engine?
- Identify assumptions for a pricing engine where component prices change over time.

System Design

- Design a scalable architecture for a bicycle pricing engine using React, Node.js, Express, and MongoDB.
- Suggest system design decisions for managing component pricing and bicycle configurations.
- How can price history be maintained while always using the latest component price for calculations?

Database Design

- Design MongoDB collections for users, components, configurations, and price history tracking.
- Create an entity relationship model for a pricing engine application.
- Suggest an efficient schema for storing bicycle configurations and reusable components.

API Design

- Design REST APIs for authentication, component management, configuration management, and pricing calculations.
- Suggest API endpoints for maintaining component price history.
- Design APIs for generating configuration pricing breakdowns.

UI/UX Design

- Suggest screens required for a pricing engine application.
- Design role-based dashboards for administrators and sales users.
- Create user flows for configuration creation and pricing calculation.

Implementation Assistance

- Generate React components for dashboard and configuration management screens.
- Create Express.js controllers and routes for component management.
- Implement JWT authentication and role-based authorization.
- Implement a pricing engine service that calculates configuration totals dynamically.
- Create MongoDB models using Mongoose.

Testing and Documentation

- Suggest test cases for pricing calculations and component management.
- Generate documentation sections for functional requirements, use cases, database design, and future enhancements.
- Create pseudocode for a dynamic pricing engine application.

AI Usage Summary

AI tools were used as a development assistant to support requirement analysis, brainstorming, documentation, code generation, and design validation. All outputs were reviewed, modified, and integrated according to the requirements of the Hero Cycles Pricing Engine assignment.